Use Cases

for

<Project>

Version 1.0 approved

Prepared by <author>

<organization>

<date created>

Revision History

Name	Date	Reason For Changes	Version

Guidance for Use Case Template

Document each use case using the template shown in the Appendix. This section provides a description of each section in the use case template.

1. Use Case Identification

1.1. Use Case ID

Give each use case a unique numeric identifier, in hierarchical form: X.Y. Related use cases can be grouped in the hierarchy. Functional requirements can be traced back to a labeled use case.

1.2. Use Case Name

State a concise, results-oriented name for the use case. These reflect the tasks the user needs to be able to accomplish using the system. Include an action verb and a noun. Some examples:

- View part number information.
- Manually mark hypertext source and establish link to target.
- Place an order for a CD with the updated software version.

1.3. Use Case History

1.3.1. Created By

Supply the name of the person who initially documented this use case.

1.3.2. Date Created

Enter the date on which the use case was initially documented.

1.3.3. Last Updated By

Supply the name of the person who performed the most recent update to the use case description.

1.3.4. Date Last Updated

Enter the date on which the use case was most recently updated.

2. Use Case Definition

2.1. Actor

An actor is a person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks. Different actors often correspond to different user classes, or roles, identified from the customer community that will use the product. Name the actor(s) that will be performing this use case.

2.2. Description

Provide a brief description of the reason for and outcome of this use case, or a high-level description of the sequence of actions and the outcome of executing the use case.

2.3. Preconditions

List any activities that must take place, or any conditions that must be true, before the use case can be started. Number each precondition. Examples:

1. User's identity has been authenticated.
2. User's computer has sufficient free memory available to launch task.

2.4. Postconditions

Describe the state of the system at the conclusion of the use case execution. Number each postcondition. Examples:

1. Document contains only valid SGML tags.
2. Price of item in database has been updated with new value.

2.5. Priority

Indicate the relative priority of implementing the functionality required to allow this use case to be executed. The priority scheme used must be the same as that used in the software requirements specification.

2.6. Frequency of Use

Estimate the number of times this use case will be performed by the actors per some appropriate unit of time.

2.7. Flow of Events

Provide a detailed description of the user actions and system responses that will take place during execution of the use case under normal, expected conditions. This dialog sequence will ultimately lead to accomplishing the goal stated in the use case name and description. This description may be written as an answer to the hypothetical question, "How do I <accomplish the task stated in the use case name>?" This is best done as a numbered list of actions performed by the actor, alternating with responses provided by the system.

2.8. Alternative Flows

Document other, legitimate usage scenarios that can take place within this use case separately in this section. State the alternative course, and describe any differences in the sequence of steps that take place. Number each alternative course using the Use Case ID as a prefix, followed by "AC" to indicate "Alternative Course". Example: X.Y.AC.1.

2.9. Exceptions

Describe any anticipated error conditions that could occur during execution of the use case, and define how the system is to respond to those conditions. Also, describe how the system is to respond if the use case execution fails for some unanticipated reason. Number each exception using the Use Case ID as a prefix, followed by "EX" to indicate "Exception". Example: X.Y.EX.1.

2.10. Includes

List any other use cases that are included ("called") by this use case. Common functionality that appears in multiple use cases can be split out into a separate use case that is included by the ones that need that common functionality.

2.11. Special Requirements

Identify any additional requirements, such as nonfunctional requirements, for the use case that may need to be addressed during design or implementation. These may include performance requirements or other quality attributes.

2.12. Assumptions

List any assumptions that were made in the analysis that led to accepting this use case into the product description and writing the use case description.

2.13. Notes and Issues

List any additional comments about this use case or any remaining open issues or TBDs (To Be Determineds) that must be resolved. Identify who will resolve each issue, the due date, and what the resolution ultimately is.

Use Case Template

Use Case ID:	UC.4		
Use Case Name:	Filter and Sort Courses		
Created By:	Gan Sui	Last Updated By:	Gan Sui
Date Created:	28/3/26	Date Last Updated:	28/3/26

Actor:	Job Seeker
Description:	This use case allows the Job Seeker to filter and sort the displayed courses based on impact level, course mode, language, and sorting criteria. The system updates the results dynamically and displays the number of matching courses.
Preconditions:	Courses are currently displayed (UC.3).
Postconditions:	1. The system displays filtered and sorted course results. 2. The system updates and displays the number of matching courses.
Priority:	High
Frequency of Use:	High

Flow of Events:	1. The Job Seeker selects one or more filters, including job impact level, course mode, and language. 2. The Job Seeker selects a sorting option such as highest rating, lowest price, or shortest duration. 3. The system applies the selected filters to the course dataset. 4. The system sorts the filtered results based on the selected sorting criteria. 5. The system updates the displayed course list within three seconds. 6. The system displays the number of courses that match the selected filters.
Alternative Flows:	UC.4.AC.1: No courses match the filters 1. The system displays a “No courses found” message. 2. The system prompts the user to adjust or remove filters.
Exceptions:	UC.4.EX.1: Invalid filter combination 1. The system detects conflicting filters. 2. The system resets filters or prompts the user to modify the selection.
Includes:	None
Special Requirements:	1. The system must update results within three seconds after filter application. 2. The system must display the number of matching courses at all times.
Assumptions:	1. Filtering operations are optimised for performance. 2. Default state displays all courses without filters.
Notes and Issues:	None